Running without public CI or internet access

Many regulated clients don't allow code or test runs on public cloud runners, and some test environments have no internet access at all. The framework is plain Node and Docker, so it can run in those places. This page lists what has to be arranged.

What needs the internet, and what to do about it

Step	Needs	Alternative inside a closed network
`npm ci`	the npm registry	An internal registry mirror. Point npm at it with `npm config set registry <url>`.
Playwright browsers	Playwright's download servers	Use the runner image (below), or download from an internal mirror with `PLAYWRIGHT_DOWNLOAD_HOST`.
The application under test	Docker images from a public registry	Pull and push the images to an internal registry first, keeping the digests pinned.
The runner base image	`mcr.microsoft.com/playwright`	Mirror it to the internal registry.
Reports	nothing at run time	The Allure and Playwright reports are static files.

The simplest route: build the runner image once

The `Dockerfile` bakes Node, the dependencies and matching browsers into one image. Build it on a machine with internet access, move it across, and run it where there is none:

```
docker build -t qa-accelerator-runner .
docker save qa-accelerator-runner -o qa-accelerator-runner.tar      # on the connected
machine
docker load -i qa-accelerator-runner.tar                            # inside the closed
network
docker run --rm --init --ipc=host --network host --env-file .env.<profile> qa-accelerator-
runner --grep @smoke
```

Record the image's digest in the validation plan, so the tooling is part of the baseline. After a run:

```
npm run evidence      # or run the same commands inside the container
```

Using a CI system other than GitHub

GitHub-hosted runners need the public internet, and GitHub Actions also needs to reach GitHub. If the client runs its own CI, such as Jenkins, GitLab or Azure DevOps on its own servers, the steps are the same:

1. Start the application under test, or point the suite at the existing test environment.
2. Run `npx playwright test` (or the runner image).
3. Run `npm run evidence` and keep the `reports/regulated/` folder as the build artifact.

JUnit results are written to `reports/_raw/junit.xml`, which every CI system can read. The pipeline file in `.github/workflows` is a worked example of these steps, not a requirement. The repository originally had Jenkins and Azure DevOps pipelines. They were removed because they couldn't be verified here. If you need one, write it from the three steps above and run it once on the client's system before relying on it.

GitHub self-hosted runners

If the client is happy to use GitHub as the control plane but wants tests to run inside its network, install a self-hosted runner there. Change `runs-on: ubuntu-latest` to the runner's label. The runner still needs outbound access to GitHub, so this is not suitable for a fully closed network.

Things to check before you rely on it

- Run once with the network disabled, and confirm nothing tries to reach the internet. Fonts, analytics and update checks in the application under test are the usual surprise.
- Confirm the browsers, fonts and the arm64 emulation step (needed only for the Toolshop demo image) are available in the closed environment. A different application will not need the emulation step.
- Confirm that no credential, personal data or production data ends up in screenshots, videos, traces or logs. These are saved on failure and are part of the evidence.
- Confirm where evidence is stored, who can read it, and how long it is kept.